

Python Programming Fundamentals Cheat Sheet

Package/Method	Description	Syntax and Code Example
AND	Returns 'True' if both statement1 and statement2 are 'True'. Otherwise, returns 'False'.	Syntax: <pre>statement1 and statement2</pre> Example: <pre>marks = 80 attendance_percentage = 85 if marks >= 60 and attendance_percentage >= 80: print("Qualified for honors") else: print("Not qualified for honors") # Output: Qualified for honors</pre>
Class Definition	Defines a blueprint for creating objects and defining their attributes and behaviors.	Syntax: <pre>class ClassName: # Class attributes and methods</pre> Example: <pre>class Person: def __init__(self, name, age): self.name = name self.age = age</pre>
Define Function	A 'function' is a reusable block of code that performs a specific task or set of tasks when called.	Syntax: <pre>def function_name(parameters): # Function body</pre> Example: <pre>def greet(name): print("Hello,", name)</pre>
Equal(==)	Checks if two values are equal.	Syntax: <pre>variable1 == variable2</pre> Example 1: <pre>x = 5</pre> returns True Example 2: <pre>age = 25 age == 30</pre> returns False
For Loop	A 'for' loop repeatedly executes a block of code for a specified number of iterations or over a sequence of elements (list, range, string, etc.).	Syntax: <pre>for variable in sequence: # Code to repeat</pre> Example 1: <pre>for num in range(1, 10): print(num)</pre> Example 2: <pre>fruits = ("apple", "banana", "orange", "grape", "kiwi") for fruit in fruits: print(fruit)</pre>
Function Call	A function call is the act of executing the code within the function using the provided arguments.	Syntax: <pre>function_name(arguments)</pre> Example: <pre>greet("Alice")</pre>
Greater Than or Equal To(>=)	Checks if the value of variable1 is greater than or equal to variable2.	Syntax: <pre>variable1 >= variable2</pre> Example 1: <pre>x = 5 and 5 >= 5</pre> returns True Example 2: <pre>quantity = 100 inventory = 150 quantity <= inventory</pre> returns True
Greater Than(>)	Checks if the value of variable1 is greater than variable2.	Syntax: <pre>variable1 > variable2</pre> Example 1: 9 > 6 returns True Example 2: <pre>age = 20 min_age = 25 age > min_age</pre> returns False
If Statement	Executes code block 'if' the condition is 'True'.	Syntax: <pre>if condition: # Code block for if statement</pre> Example: <pre>if temperature > 30: print("It's a hot day!")</pre>

<code>if Else Else</code>	Executes the first code block if condition1 is "True", otherwise checks condition2, and so on. If no condition is "True", the else block is executed.	Syntax: <pre>if condition1: # Code if condition1 is True elif condition2: # Code if condition2 is True else: # Code if no condition is True</pre> Example: <pre>score = 85 # Example score if score >= 90: print("You got an A!") elif score >= 80: print("You got a B.") else: print("You need to work harder.") # Output: "You got a B."</pre>
<code>if Else Statement</code>	Executes the first code block if the condition is "True", otherwise the second block.	Syntax: <pre>if condition: # Code, if condition is True else: # Code, if condition is False</pre> Example: <pre>if age >= 18: print("You're an adult.") else: print("You're not an adult yet.")</pre>
<code>Less Than or Equal To(<=)</code>	Checks if the value of variable1 is less than or equal to variable2.	Syntax: <pre>variable1 <= variable2</pre> Example 1: <pre>5 <= 5 and 3 <= 5</pre> returns True Example 2: <pre>size = 20 max_size = 50 size <= max_size</pre> returns True
<code>Less Than(<)</code>	Checks if the value of variable1 is less than variable2.	Syntax: <pre>variable1 < variable2</pre> Example 1: <pre>4 < 5</pre> returns True Example 2: <pre>score = 88 minimum_score = 85 score > minimum_score</pre> returns True
<code>Loop Controls</code>	<code>break</code> exits the loop prematurely. <code>continue</code> skips the rest of the current iteration and moves to the next iteration.	Syntax: <pre>for: # Code to repeat if: # Condition statement break for: # Code to repeat if: # Condition statement continue</pre> Example 1: <pre>for num in range(1, 6): if num == 3: break print(num)</pre> Example 2: <pre>for num in range(1, 6): if num == 3: continue print(num)</pre>
<code>NOT</code>	Returns "True" if variable is "False", and vice versa.	Syntax: <pre>not variable</pre> Example: <pre>is_adult = False not(is_adult)</pre> returns True if the variable is False (i.e., unchecked).
<code>Not Equal(!=)</code>	Checks if two values are not equal.	Syntax: <pre>variable1 != variable2</pre> Example: <pre>5 != 10 4 != 5</pre> returns True Example 2: <pre>count = 0 count != 5</pre> returns False
<code>Object Creation</code>	Creates an instance of a class (object) using the class constructor.	Syntax: <pre>Object_name = Classname(argument)</pre> Example: <pre>person = Person("Alice", 25)</pre>

OR	Returns 'True' if either statement1 or statement2 (or both) are 'True'. Otherwise, returns 'False'.	Syntax: <pre>statement1 or statement2</pre> Example: <pre>'Passes! Party Invitation' grade = 10 if grade == 10 or grade == 12: print('Passes! Party Invitation') else: print('Not eligible')</pre> returns True
range()	Generates a sequence of numbers within a specified range.	Syntax: <pre>range(stop) range(start, stop) range(start, stop, step)</pre> Example: <pre>range() Generates a sequence of integers from 0 to 4. range(5) Generates a sequence of integers from 0 to 4. range(2, 10) Generates all integers from 2 to 9. range(2, 11, 2) Generates odd integers from 1 to 9.</pre>
Return Statement	'Return' is a keyword used to send a value back from a function to its caller.	Syntax: <pre>return value</pre> Example: <pre>def add(a, b): return a + b result = add(3, 5)</pre>
Try-Except Block	Tries to execute the code in the try block. If an exception of the specified type occurs, the code in the except block is executed.	Syntax: <pre>try: # Code that might raise an exception except exceptType: # Code to handle the exception</pre> Example: <pre>try: num = int(input("Enter a number: ")) except ValueError: print("Invalid input. Please enter a valid number.")</pre>
Try-Except with Else Block	Code in the 'else' block is executed if no exception occurs in the try block.	Syntax: <pre>try: # Code that might raise an exception except # Code that might raise an exception except exceptType: # Code to handle the exception else: # Code to execute if no exception occurs</pre> Example: <pre>try: num = int(input("Enter a number: ")) except ValueError: print("Invalid input. Please enter a valid number.") else: print("The entered:", num)</pre>
Try-Except with Finally Block	Code in the 'finally' block always executes, regardless of whether an exception occurred.	Syntax: <pre>try: # Code that might raise an exception except exceptType: # Code to handle the exception finally: # Code that always executes</pre> Example: <pre>try: file = open("data.txt", "r") data = file.read() except: print("Error reading file") finally: file.close()</pre>
While Loop	A 'while' loop repeatedly executes a block of code as long as a specified condition remains 'True'.	Syntax: <pre>while condition: # Code to repeat</pre> Example: <pre>count = 0 while count < 5: print(count) count += 1</pre>